

00 — SUT RECON (Phase 0)

- **Sinh viên:** Ninh Văn Khải — MSSV **23127060**
- **SUT:** EShop (`eshop-sut/`) — Express 5 + SQLite + React/Vite
- **Thời điểm recon:** 2026-08-22 (giờ máy thật, Asia/Ho_Chi_Minh)
- **Feature phụ trách:** FR-03 (Quên/Đặt lại mật khẩu) · FR-08 (Thanh toán) · FR-15 (Quản lý sản phẩm)

Tài liệu này là **input bắt buộc** cho mọi Phase sau. Mọi selector và expected result trong test đều phải

dẫn nguồn về một dòng trong bảng dưới đây. Không đoán selector.

1. Môi trường đã xác minh

Thành phần	Lệnh xác minh	Kết quả thật
Backend Express	<code>curl http://localhost:3000/api/products</code>	200 + JSON 5 sản phẩm seed
Frontend-web (Vite)	<code>curl -o /dev/null -w "%{http_code}" http://localhost:5173/</code>	200
Frontend-admin (Vite)	<code>Get-NetTCPConnection -State Listen</code>	Ban đầu chưa chạy → đã <code>npm install</code> cho frontend-admin (thiếu <code>node_modules</code>), 😬 Khải cần chạy <code>npm run dev</code> để Vite cấp port (dự kiến 5174)
Node	<code>node -v</code>	v22.20.0
Playwright	<code>npx playwright --version</code>	1.62.1

Ghi chú port: `run_servers.sh` không chỉ định port cho Vite → Vite tự cấp tăng dần. Web đang giữ **5173**;

admin sẽ nhận **5174** nếu 5173 bận. Giá trị thật đọc từ ENV trong `automation/tests/utils/env.js`

(`WEB_BASE_URL` , `ADMIN_BASE_URL` , `API_BASE_URL`) nên đổi port không phải sửa code test.

Backend port 3000 là bắt buộc, không đổi được: cả `frontend-web` và `frontend-admin` hardcode

`http://localhost:3000/api` trong source (`ForgotPassword.jsx:17` , `Checkout.jsx:30` , `App.jsx:4`).

2. Seed data (DROP + seed lại mỗi lần khởi động backend — `database.js:14-20`)

Bảng	Nội dung seed
users	admin@eshop.com / Admin123! (role admin), test@eshop.com / Test1234! (role user)
categories	1 Điện thoại · 2 Laptop · 3 Phụ kiện
products	id 1..5 — iPhone 15 Pro Max 30tr · Galaxy S24 Ultra 28tr · MacBook Pro M3 45tr · AirPods Pro 2 6tr · Keychron Q1 4tr
coupons	SAVE10 percent 10 min 300k · BIGBUY fixed 50k min 500k · VIP100 fixed 100k min 300k max 2 lượt · EXPIRED percent 20 hết hạn 2020-01-01

Hệ quả cho thiết kế test:

1. Restart backend = mất sạch dữ liệu test → test **không được** phụ thuộc dữ liệu do test khác tạo.
2. Mật khẩu lưu **plaintext** (database.js:88-90 , server.js:23) → không hash.
3. Mỗi test tự tạo user riêng bằng email random ts-<timestamp>-<rand>@eshop.test .
4. Giỏ hàng backend (userCarts) là biến **in-memory** (server.js:14) → mất khi restart, không dedupe, không trữ kho.

3. Bảng route ↔ selector ↔ API ↔ rủi ro

3.1 FR-03 — Quên mật khẩu & Đặt lại mật khẩu

Route FE: /forgot-password (frontend-web/src/pages/ForgotPassword.jsx) — 2 bước trong **cùng 1 route**, đổi bằng state step .

Bước	Selector (nguồn)	API gọi	Rủi ro / bug candidate
B1 nhập email	getByRole('textbox') — input type=text , <label> KHÔNG có htmlFor (dòng 47-53) ⇒ getByLabel() không dùng được	POST /api/forgot-password	Trả resetToken thẳng trong response body
B1 submit	getByRole('button', { name: 'Lấy mã OTP' }) (dòng 57)	—	Không rate-limit
B2 hộp OTP	.bg-green-100 chứa Mã OTP của bạn là: <token> (dòng 68-70) → lấy token bằng regex /\d{4}/ trên text	—	Token lộ trên UI
B2 nhập OTP	textbox thứ nhất của step 2 (dòng 73-79)	—	Token chỉ 4 chữ số ⇒ 9000 tổ hợp
B2 mật khẩu mới	page.locator('input[type="password"]') (dòng 83-89)	—	—
B2 submit	getByRole('button', { name: 'Đặt lại mật khẩu' }) (dòng 93)	POST /api/reset-password	Không kiểm tra policy mật khẩu ở backend
Kết quả	alert() cho cả thành công lẫn lỗi ⇒ bắt bằng page.on('dialog') (dòng 22, 33, 36)	—	Không có toast DOM để assert

● **BẦY FE quan trọng (ForgotPassword.jsx:26):**

```
const flawedStrongPasswordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*\s)[A-Za-z\d\s]{8,}$/;
```

- Thông báo lỗi nói "*phải có KÝ TỰ ĐẶC BIỆT*" nhưng regex thực tế đòi **khoảng trắng** (`(?=.*\s)`)

và **cấm** mọi ký tự đặc biệt (character class chỉ `[A-Za-z\d\s]`).

- $\Rightarrow$ `NewPass123!` (mạnh, có ký tự đặc biệt) **BỊ CHẶN**; `New Pass 123` (yếu hơn, có space) **ĐƯỢC CHẤP NHẬN**.
- Đây là **BUG-03-01** (mismatch giữa message và validation) — sẽ được cover bởi test case FR03.

Hành vi backend đã xác minh bằng curl:

```
POST /api/forgot-password {"email":"test@eshop.com"}
 $\Rightarrow$  200 {"message":"Mã đặt lại mật khẩu đã được tạo","resetToken":"5860"}  $\leftarrow$  token LỘ

POST /api/forgot-password {"email":"nobody-xyz@nope.com"}
 $\Rightarrow$  404 {"error":"User not found"}  $\leftarrow$  user enumeration

POST /api/reset-password {"email":"test@eshop.com","resetToken":"0000",...}
 $\Rightarrow$  400 {"error":"Invalid token or email"}
```

Bug candidate FR-03: (a) token lộ trong HTTP response; (b) token 4 số $\rightarrow$ brute-force được;

(c) token không có expiry (`server.js:66-82` không lưu thời điểm tạo); (d) không rate-limit;

(e) user enumeration qua 404; (f) backend không kiểm tra độ mạnh mật khẩu (đặt được `1`);

(g) mật khẩu plaintext; (h) reset **không** xóa `locked_until` $\Rightarrow$ đổi mật khẩu xong vẫn không đăng nhập được;

(i) regex FE sai chuẩn (BẦY ở trên).

3.2 FR-08 — Thanh toán (Checkout)

Route FE: `/` $\rightarrow$ `/cart` $\rightarrow$ `/checkout` (`Home.jsx`, `Cart.jsx`, `Checkout.jsx`, `context/CartContext.jsx`).

Màn	Selector (nguồn)	API gọi	Rủi ro / bug candidate
Home — thêm giỏ	<code>getByRole('button', { name: 'Thêm vào giỏ' })</code> trong card sản phẩm (Home.jsx:110)	— (client-side)	—
Cart trống	<code>getByText('Giỏ hàng của bạn đang trống')</code> (Cart.jsx:24)	—	—
Cart → checkout	<code>getByRole('button', { name: 'Tiến hành thanh toán' })</code> (Cart.jsx:74)	—	Chưa login ⇒ <code>alert()</code> + redirect <code>/login</code>
Checkout — tổng tiền	<code>getByRole('spinbutton')</code> — input <code>type=number</code> <code>editableTotal</code> SỬA ĐƯỢC (Checkout.jsx:96-106)	—	● Price tampering
Checkout — coupon	<code>getByPlaceholder('Nhập mã giảm giá...')</code> (Checkout.jsx:115)	POST <code>/api/apply-coupon</code>	● công thức percent sai
Checkout — áp dụng	<code>getByRole('button', { name: 'Áp dụng' })</code> (Checkout.jsx:120)	—	—
Checkout — xác nhận	<code>getByRole('button', { name: 'Xác Nhận Thanh Toán' })</code> (Checkout.jsx:146)	POST <code>/api/checkout</code>	Backend tin <code>total_amount</code> client
Thành công	<code>getByRole('heading', { name: 'Thanh toán thành công!' })</code> (Checkout.jsx:70)	—	—

● Ba bug lớn đã xác minh bằng curl:

```
# (1) Price tampering – backend tin tuyệt đối total_amount của client
POST /api/checkout Authorization: Bearer <user token>
{"total_amount":1,"shipping_address":null}
⇒ 200 {"message":"Checkout successful","orderId":1}

# (2) Coupon percent tính SAI DẤU – server.js:432 dùng (1 - discount_value) thay vì (discount_value/
POST /api/apply-coupon {"code":"SAVE10","total_amount":1000000}
⇒ 200 {"discount_amount":-9000000,"final_amount":1000000}
  ← giảm 10% nhưng tổng tiền TĂNG GẤP 10 LẦN

# (3) IDOR – GET /api/orders/:id KHÔNG có authenticateToken (server.js:340)
GET /api/orders/1 (không gửi header nào)
⇒ 200 {"id":1,"user_id":2,"total_amount":1,"status":"pending",...}
```

Bug candidate FR-08 khác (đọc từ source, cần test xác nhận):

- `clearCart` được import ở Checkout.jsx:8 nhưng **không bao giờ được gọi** ⇒ giỏ không xóa sau thanh toán ⇒ double-checkout.
- FE gửi `items: cart` nhưng backend `/api/checkout` (server.js:302-317) **bỏ qua** `items`, chỉ lưu `total_amount` + `shipping_address`; mà FE **không gửi** `shipping_address` ⇒ đơn hàng luôn có địa chỉ `null`.
- Không validate giỏ trống, `total_amount` âm, `total_amount` không phải số.
- Không trừ tồn kho (schema `products` không có cột stock).

- Cart lưu trong **React Context (in-memory)**, KHÔNG localStorage (CartContext.jsx:7) ⇒ reload trang là mất giỏ ⇒ **test phải add sản phẩm qua UI trong cùng phiên**, không seed qua API cho luồng UI.

3.3 FR-15 — Quản lý sản phẩm (Admin)

Route FE: frontend-admin là **SPA 1 file** (src/App.jsx , 922 dòng), đổi màn bằng state activeTab , không có router.

Bước	Selector (nguồn)	API gọi	Rủi ro
Login admin	getByPlaceholder('Email') · getByPlaceholder('Password') · getByRole('button', { name: 'Login' }) (App.jsx:185-211)	POST /api/login	FE chặn role !== 'admin' bằng alert('Bạn không phải là admin!') (dòng 65)
Token	localStorage['adminToken'] (dòng 68)	—	Có thể inject qua addInitScript để bỏ qua login UI
Vào tab SP	getByText('Sản phẩm', { exact: true }) — là không phải button (dòng 238-244)	—	—
Form thêm/sửa	getByPlaceholder('Tên sản phẩm') (required) · getByPlaceholder('Giá tiền') (type=number) · getByPlaceholder('URL Ảnh') · getByPlaceholder('Mô tả') (textarea) · getByRole('combobox') (App.jsx:490-540)	—	Giá tiền không required
Submit	getByRole('button', { name: 'Lưu sản phẩm' }) (dòng 545)	POST hoặc PUT /api/products	—
Tiêu đề form	getByText('Thêm sản phẩm mới') ↔ getByText('Sửa sản phẩm') (dòng 486-488)	—	Phân biệt chế độ add/edit
Bảng SP	getByRole('row').filter({ hasText: '<tên>' }) ; trong dòng có nút Sửa / Xóa (dòng 570-600)	DELETE /api/products/:id	—
Thông báo	Thêm mới: không alert , chỉ refetch. Sửa: alert('Cập nhật thành công!').Lỗi: alert('Lỗi lưu sản phẩm: ...')	—	Bắt bằng page.on('dialog')

● **Bug lớn FR-15 đã xác minh:**

```
# (1) Thiếu access control HOÀN TOÀN – POST/PUT/DELETE /api/products không có authenticateToken
POST /api/products (KHÔNG gửi Authorization)
  {"name":"RECON-NOAUTH","price":-5,...}
⇒ 200 {"message":"Product created","id":6} ← tạo được, giá ÂM cũng insert

DELETE /api/products/6 (KHÔNG gửi Authorization)
⇒ 200 {"message":"Product deleted"}

# (2) DELETE id không tồn tại vẫn báo thành công (server.js:186-191 không check this.changes)
DELETE /api/products/99999
⇒ 200 {"message":"Product deleted"}

# (3) Kiểu dữ liệu bất nhất – id CHẴN trả price dạng STRING (server.js:158)
GET /api/products/2 ⇒ {"id":2,...,"price":"28000000"} ← string
GET /api/products/1 ⇒ {"id":1,...,"price":30000000} ← number

# (4) id không tồn tại trả 200 {} thay vì 404 (server.js:157)
GET /api/products/9999 ⇒ 200 {}
```

● BUG "fake mass update" (App.jsx:107-112) — bug FE nặng nhất của FR-15:

```
await axios.put(`${API_URL}/products/${productForm.id}`, productForm);
const fakeMassUpdatedProducts = products.map((p) => ({
  ...p,
  name: productForm.name, // ← gán tên SP đang sửa cho TẤT CẢ sản phẩm
}));
setProducts(fakeMassUpdatedProducts);
```

Sau khi bấm **Lưu sản phẩm** ở chế độ sửa, **cả bảng đổi thành cùng 1 tên** cho tới khi reload.

Assertion bắt bug: đếm số dòng có tên mới > 1.

Bug candidate FR-15 khác: tạo SP `name` rỗng (backend không validate); `price` là chuỗi/âm/cực lớn vẫn insert;

`category_id` không tồn tại vẫn insert (không có FOREIGN KEY); XSS trong `name` / `description`

(lưu ý: bảng admin render bằng `{p.name}` — React **có** escape ⇒ XSS không thực thi ở đây, nhưng payload vẫn lưu được vào DB

và render `dangerouslySetInnerHTML` ở nơi khác — App.jsx:801 cho `shipping_address`, Home.jsx:64 cho ô tìm kiếm).

4. Ngoài phạm vi (đọc để tránh, KHÔNG viết test)

Chức năng	Vì sao ngoài phạm vi
Import CSV <code>/api/admin/import-products</code>	Thuộc FR-16 — feature của thành viên khác
Quản lý danh mục / coupon / user (admin)	Thuộc FR-14 và các FR khác
Đăng ký / Đăng nhập (<code>/register</code> , <code>/login</code>)	FR-01 / FR-02 — chỉ dùng làm tiền đề cho FR-03 và FR-08, không viết test case tính điểm
<code>frontend-mobile</code> (FR-20)	Đề §5: Pool D không tính điểm HW04

Cảnh báo tiền đề (`Login.jsx`): trang login của web có nhiều lỗi hiển thị (heading ghi "**Đăng Ký**", label ghi "**Username**", ô mật khẩu là `type="text"`). Nút submit là **Sign In** , KHÔNG phải **Đăng nhập** .

⇒ Page Object login phải dùng `getByRole('button', { name: 'Sign In' })` và định vị 2 ô bằng **thứ tự textbox**,

không dùng `input[type=password]` (không tồn tại ở trang này).

Cảnh báo `ProductDetail.jsx:22-25` : nút "Thêm vào giỏ hàng" ở trang chi tiết **bỏ qua click đầu tiên**

(`clickCount === 0 → return`). ⇒ Luồng FR-08 phải thêm sản phẩm từ **Home**, không qua `ProductDetail`.

Cảnh báo `/api/login` (`server.js:53`): mỗi lần sai mật khẩu, `login_attempts` tăng **2** (không phải 1)

⇒ chỉ cần **2 lần** sai là bị khoá 3 phút. Test FR-03 phải tránh đăng nhập sai ngoài ý muốn.

5. Kết luận Phase 0

- ✓ Backend `3000` và frontend-web `5173` đã sống, xác minh bằng `curl` .
- ✓ Đã xác minh **bằng request thật** 9 hành vi lỗi: token lộ, user enumeration, price tampering, coupon nghịch dấu,

IDOR order, product CRUD không auth, DELETE id ảo báo thành công, price string với id chẵn, id lạ trả `200 {}` .

- ✓ Toàn bộ selector trong bảng §3 **đọc trực tiếp từ JSX**, có số dòng dẫn nguồn.
- ⚠ frontend-admin thiếu `node_modules` → đã cài; 🤖 **Khởi chạy `run_servers.sh` và dán output xác nhận port admin thật.**

→ **Sẵn sàng sang Phase 1: thiết kế bảng test case ≥12 TC cho từng feature.**